4190.307 / FALL 2026

xv6 + PA1–PA3 코드 읽기

1부를 제외한 독립형 2부 · 스마트폰에서 읽기 쉬운 판형 · 제출 코드와 기출형 해설 수록

이 노트가 답하려는 질문

- 트랩 직후에는 누가 어떤 레지스터와 페이지 테이블을 바꾸는가?
- fork/wait/sched/sleep의 상태 전이는 코드에서 어디에 나타나는가?
- Sv39 주소 변환과 lazy page fault는 어떻게 연결되는가?
- 내 PA1–PA3 코드가 일반 xv6 경로를 어떻게 바꾸는가?

최근 기출이 보여 준 우선순위

공유된 2023-2·2024-1·2024-2·2025-2 중간고사 4회 모두에서 trap/syscall/xv6 코드 읽기와 주소 변환·VM 장문이 확인됐다. 스케줄 타임라인도 3회 등장했다. 이번 문서는 이 형식을 연습 문제로 재구성했다.

과거 PA의 이름이 같아도 구현 주제는 다를 수 있다. 특히 2025-2의 PA2는 IPI, 2024-2의 PA3는 SNULE다. 아래 PA2(PMP)·PA3(EDF)의 답은 이번 학기 제출 브랜치 기준이다.

코드 박스는 해당 Git 브랜치의 실제 파일에서 발췌했다. 공간을 위해 주석·빈 줄을 일부 생략했고, …는 중간 생략, 들여쓴 이어지는 줄은 지면용 줄바꿈이다. 풀이에는 생략 부분에 의존하는 주장을 넣지 않았다.

차례

01 기출형 핵심 지도 → 02 트랩·시스템 콜 → 03 프로세스·스케줄링 → 04 주소 변환·메모리 → 05 PA1 → 06 PA2 → 07 PA3 → 08 예상 문제·해설 → 09 3분 카드

01 · 먼저 그리는 전체 지도

시험장에서 순서를 잃지 않기 위한 1쪽짜리 실행 경로

사용자 시스템 콜

user stub(a7=번호, a0-a5=인자) → ecall → 하드웨어가 S-mode·sepc/scause/sstatus 설정 → stvec의 uservec(아직 user page table) → trapframe 저장·kernel satp 전환 → usertrap() → syscall() → trapframe->a0 반환값 → prepare_return() → userret → sret.

타이머가 스케줄러를 부를 때

timer interrupt → devintr()가 2 반환 → usertrap()/kerneltrap()의 yield() → p->state=RUNNABLE → sched() → swtch() → scheduler() → 다음 RUNNABLE 선택. PA3에서는 먼저 RT의 deadline을 검사하고, 없으면 일반 RR을 쓴다.

메모리에 처음 접근할 때

VA = VPN[2]|VPN[1]|VPN[0]|offset → satp가 가리킨 3단계 page table을 walk → leaf PTE의 PPN + offset → PA. PTE가 없고 유효한 lazy 범위면 page fault(scause 13/15) → vmfault()가 새 0 페이지를 매핑 → 같은 명령 재실행.

기출에서 반복된 코드 읽기

- 2023-2 p.2 Q2: 하드웨어가 자동 수행하는 일과 uservec 소프트웨어의 일을 분리.
- 2024-2 p.2 Q2: uservec 첫 csrw와 sscratch, stvec, sepc의 값을 추적.
- 2023-2 p.6 Q6: per-process / per-core / per-system 자료구조 분류.
- 2024-1 p.5 Q4: EDF 작업의 release·deadline을 놓고 타임라인 작성.

• 2025-2 p.6 Q5: a7과 a0, M-mode trap의 PC 전진·pending 처리 구분.

02 · 트랩: 하드웨어와 소프트웨어

기출에서 가장 직접적으로 코드 실행 순서를 묻는 영역

ecall 직후 하드웨어가 한 일

U→S trap에서는 특권이 S로 바뀌고 PC가 stvec으로 간다. 이전 PC는 sepc, 원인은 scause에 기록된다. SIE는 꺼진다. 그러나 하드웨어는 사용자 레지스터를 trapframe에 저장하거나, sp를 커널 스택으로 바꾸거나, satp를 커널 페이지 테이블로 바꾸지 않는다. 이 셋은 xv6 코드의 일이다.

ecall의 sepc는 ecall 명령 주소다. +4는 usertrap()에서 수행한다. 타이머 인터럽트의 sepc는 중단된 명령의 주소이며 임의로 +4 하면 안 된다.

uservec 첫 두 명령: a0의 의미가 바뀐다

사용자 a0 임시 보존 | pa3:kernel/trampoline.S L30-38

```
        csrw sscratch, a0
        li a0, TRAPFRAME
```

첫 csrw 이전 a0는 사용자 레지스터 값이다. csrw가 그 값을 sscratch에 보관한다. 이어 a0=TRAPFRAME 가상주소로 바꿔 레지스터 저장에 쓰고, 뒤에서 sscratch를 읽어 원래 a0도 trapframe에 저장한다. 첫 csrw 직전 sscratch의 이전 값에 별도 의미를 가정하면 안 된다.

trapframe에서 커널 문맥을 읽고 satp 전환 | pa3:kernel/trampoline.S L70-98

```
        csrr t0, sscratch
        sd t0, 112(a0)
        ld sp, 8(a0)
        ld tp, 32(a0)
        ld t0, 16(a0)
        ld t1, 0(a0)
```

```
        sfence.vma zero, zero
        csrw satp, t1
        sfence.vma zero, zero
        jalr t0
```

TRAMPOLINE과 TRAPFRAME은 사용자 페이지 테이블에 높은 가상주소로 매핑된다. uservec는 커널 페이지 테이블로 전환하기 전에도 실행돼야 하므로 trampoline 코드가 양쪽 페이지 테이블에서 같은 VA에 있다. 커널 sp/tp/trap 주소/satp 값은 trapframe의 kernel_* 필드에서 읽는다.

C 코드: syscall·page fault·timer 분기

usertrap()의 세 가지 주요 분기 | pa3:kernel/trap.c L49-75, 81-94

```
  struct proc *p = myproc();
  p->trapframe->epc = r_sepc();
  if (r_scause() == 8) {
    if (killed(p))
      kexit(-1);
    p->trapframe->epc += 4;
    intr_on();
    syscall();
  } else if ((which_dev = devintr()) != 0) {
  } else if ((r_scause() == 15 || r_scause() == 13) &&
             vmfault(p->pagetable, p->sz, r_stval(),
                     (r_scause() == 13) ? 1 : 0) != 0) {
  } else {
    ...
  if (killed(p))
    kexit(-1);
  if (which_dev == 2)
    yield();
  prepare_return();
  uint64 satp = MAKE_SATP(p->pagetable);
  return satp;
```

scause=8은 U-mode ecall이다. epc를 4 전진시키고 syscall()을 호출한다. scause=13/15는 load/store page fault이며 vmfault()가 복구할 수 있다. devintr()==2는 timer라서 yield()한다. 예외를 처리할 수 없으면 프로세스를 killed로 표시한다.

prepare_return(): 복귀 CSR과 다음 trap 준비 | pa3:kernel/trap.c L101-132

```
prepare_return(void)
{
  struct proc *p = myproc();
  intr_off();
  uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
  w_stvec(trampoline_uservec);
  p->trapframe->kernel_satp = r_satp();
  p->trapframe->kernel_sp = p->kstack + PGSIZE;
  p->trapframe->kernel_trap = (uint64)usertrap;
  p->trapframe->kernel_hartid = r_tp();
  unsigned long x = r_sstatus();
  x &= ~SSTATUS_SPP;
  x |= SSTATUS_SPIE;
  w_sstatus(x);
  w_sepc(p->trapframe->epc);
}
```

stvec는 다시 uservec로, SPP=0은 복귀 목적 U-mode, SPIE=1은 U-mode에서 인터럽트 허용을 뜻한다. sepc에는 수정된 p->trapframe->epc를 쓴다. 이어 userret이 user satp와 사용자 레지스터를 복원하고 sret한다.

userret(): user satp, a0, sret | pa3:kernel/trampoline.S L100-114, 148-153

```
.globl userret
userret:
        fence.i
        sfence.vma zero, zero
        csrw satp, a0
        sfence.vma zero, zero
        li a0, TRAPFRAME
    ...
        ld a0, 112(a0)
```

```
        sret
```

시스템 콜 번호·인자·반환값

사용자 stub의 세 명령 | pa3:user/usys.pl L19–21

```
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
```

trapframe a0–a5 인자, a7 번호, a0 반환 | pa3:kernel/syscall.c L34–57, 146–160

```
static uint64
argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0:
    return p->trapframe->a0;
  case 1:
    return p->trapframe->a1;
  case 2:
    return p->trapframe->a2;
  case 3:
    return p->trapframe->a3;
  case 4:
    return p->trapframe->a4;
  case 5:
    return p->trapframe->a5;
  }
  panic("argraw");
  return -1;
}
void
    ...
void
syscall(void)
{
  int num;
  struct proc *p = myproc();
  num = p->trapframe->a7;
  if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
```

```
  } else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
```

사용자 함수의 첫 인자는 a0다. 시스템 콜 번호는 a7이다. syscall()은 trapframe->a7로 함수 표를 찾고, 결과를 trapframe->a0에 덮어쓴다. 따라서 userret에서 복원되는 a0가 사용자에게 보이는 반환값이다.

U-mode ecall과 S-mode ecall 비교

- U→S: scause=8, stvec=uservec, sepc는 usertrap에서 +4, 마지막은 sret.
- S→M(PA2): mcause=9, mtvec=machinevec, mepc는 machinevec에서 +4, 마지막은 mret.
- 비동기 인터럽트: 중단 명령을 재개하므로 PC를 무조건 +4 하지 않는다. 원인 장치의 pending도 필요한 방식으로 해제해야 재진입을 피한다.

2025-2 기출의 M software interrupt 문제는 이번 PA2의 S-mode ecall과 다른 원인이다. 'M-mode trap'이라는 공통점만으로 동일한 첫 명령·pending 처리를 답하면 안 된다.

03 · 프로세스, 문맥 교환, 잠들기

fork/wait 상태 추적과 자료구조 범위 분류 대비

누가 소유하는 자료인가

- 프로세스별: struct proc, pagetable, trapframe, kernel stack, context, sz, ofile/cwd. 한 프로세스의 사용자 상태와 커널 실행 재개 지점을 보유한다.
- CPU별: struct cpu의 현재 proc와 scheduler context, noff/intena. 같은 CPU에서 다음에 무엇을 돌릴지 기억한다.
- 시스템 전체: proc[NPROC], cpus[NCPU], kernel_pagetable, trampoline 코드, ticks, syscalls[] 함수 표.

struct cpu와 struct proc 핵심 필드 | pa3:kernel/proc.h L21-27, 79-108

```
struct cpu {
  struct proc *proc;
  struct context context;
  int noff;
  int intena;
};
    ...
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING,
    ZOMBIE };
struct proc {
  struct spinlock lock;
  enum procstate state;
  void *chan;
  int killed;
  int xstate;
  int pid;
  int runtime;
  int period;
  uint deadline;
  struct proc *parent;
  uint64 kstack;
  uint64 sz;
  pagetable_t pagetable;
  struct trapframe *trapframe;
  struct context context;
  struct file *ofile[NOFILE];
  struct inode *cwd;
  char name[16];
};
```

trapframe은 U→S 경계에서 사용자 레지스터를 보존한다. context는 swtch()가 커널 실행 스레드의 callee-saved 레지스터를 보존한다. 둘을 같은 문맥이라고 답하면 저장 시점과 목적을 놓친다.

fork(): 부모와 자식은 어디서 갈라지는가

kfork()의 복사와 두 반환값 | pa3:kernel/proc.c L274-308

```
  if (uvmcopy(p->pagetable, np->pagetable, p->sz) < 0) {
    freeproc(np);
    release(&np->lock);
```

```
    return -1;
  }
  np->sz = p->sz;
  *(np->trapframe) = *(p->trapframe);
  np->trapframe->a0 = 0;
  for (i = 0; i < NOFILE; i++)
    if (p->ofile[i])
      np->ofile[i] = filedup(p->ofile[i]);
  np->cwd = idup(p->cwd);
  safestrcpy(np->name, p->name, sizeof(p->name));
  pid = np->pid;
  release(&np->lock);
  acquire(&wait_lock);
  np->parent = p;
  release(&wait_lock);
  acquire(&np->lock);
  np->state = RUNNABLE;
  release(&np->lock);
  return pid;
```

uvmcopy는 현재 존재하는 사용자 페이지를 새 물리 페이지에 복사한다(COW 아님). trapframe을 복사한 뒤 자식의 a0=0으로 고친다. 부모는 kfork()가 반환한 자식 PID를 syscall()의 a0로 받는다. 파일 디스크립터는 filedup으로 같은 file 구조의 참조 수를 늘린다. 자식은 마지막에 RUNNABLE이 된다.

exit/wait와 ZOMBIE

ZOMBIE 전환과 부모의 자식 회수 | pa3:kernel/proc.c L351–368, 380–421

```
  acquire(&wait_lock);
  reparent(p);
  wakeup(p->parent);
  acquire(&p->lock);
  p->xstate = status;
  p->state = ZOMBIE;
  release(&wait_lock);
  sched();
  panic("zombie exit");
    ...
```

```
  acquire(&wait_lock);
  for (;;) {
    havekids = 0;
    for (pp = proc; pp < &proc[NPROC]; pp++) {
      if (pp->parent == p) {
        acquire(&pp->lock);
        havekids = 1;
        if (pp->state == ZOMBIE) {
          pid = pp->pid;
          if (addr != 0 &&
              copyout(p->pagetable, p->sz, addr, (char
    *)&pp->xstate,
                      sizeof(pp->xstate)) < 0) {
            release(&pp->lock);
            release(&wait_lock);
            return -1;
          }
          pp->parent = 0;
          freeproc(pp);
          release(&pp->lock);
          release(&wait_lock);
          return pid;
        }
        release(&pp->lock);
      }
    }
    if (!havekids || killed(p)) {
      release(&wait_lock);
      return -1;
    }
    sleep_prepare(p);
    release(&wait_lock);
    sleep();
    acquire(&wait_lock);
```

자식의 exit은 즉시 proc 슬롯을 지우지 않고 ZOMBIE로 남긴다. 부모의 wait가 자식을 찾아 상태를 회수하고 freeproc한다. 아직 종료한 자식이 없으면 부모는 잠들고, 자식 종료 시 wakeup으로 다시 검사한다. 부모가 wait에 들어갈 때와 자식이 종료할 때의 순서 경합을 lock과 채널 등록으로 막는다.

swtch(), sched(), yield()

swtch: ra/sp/s0–s11의 저장·복원 | pa3:kernel/swtch.S L8–23, 25–40

```
.globl swtch
swtch:
        sd ra, 0(a0)
        sd sp, 8(a0)
        sd s0, 16(a0)
        sd s1, 24(a0)
        sd s2, 32(a0)
        sd s3, 40(a0)
        sd s4, 48(a0)
        sd s5, 56(a0)
        sd s6, 64(a0)
        sd s7, 72(a0)
        sd s8, 80(a0)
        sd s9, 88(a0)
        sd s10, 96(a0)
        sd s11, 104(a0)
    ...
        ld ra, 0(a1)
        ld sp, 8(a1)
        ld s0, 16(a1)
        ld s1, 24(a1)
        ld s2, 32(a1)
        ld s3, 40(a1)
        ld s4, 48(a1)
        ld s5, 56(a1)
        ld s6, 64(a1)
        ld s7, 72(a1)
        ld s8, 80(a1)
        ld s9, 88(a1)
        ld s10, 96(a1)
        ld s11, 104(a1)
        ret
```

sched의 전제와 일반 yield | pa3:kernel/proc.c L543–571

```
sched(void)
{
  int intena;
  struct proc *p = myproc();
  if (!holding(&p->lock))
```

```
    panic("sched p->lock");
  if (mycpu()->noff != 1)
    panic("sched locks");
  if (p->state == RUNNING)
    panic("sched RUNNING");
  if (intr_get())
    panic("sched interruptible");
  intena = mycpu()->intena;
  swtch(&p->context, &mycpu()->context);
  mycpu()->intena = intena;
}
void
yield(void)
{
  struct proc *p = myproc();
  acquire(&p->lock);
  p->state = RUNNABLE;
  sched();
  release(&p->lock);
}
```

yield는 프로세스를 RUNNABLE로 만들고 sched를 호출한다. sched는 p->lock을 보유하고 RUNNING이 아닌 상태여야 하며, 인터럽트가 꺼져 있어야 한다. swtch(&p->context, &cpu->context)는 현재 프로세스 커널 문맥을 저장하고 scheduler 문맥을 복원한다. 사용자 레지스터 저장은 앞의 trapframe 경로다.

이 저장소의 sleep_prepare → sleep

채널 선등록과 wakeup | pa3:kernel/proc.c L611–659

```
sleep_prepare(void *chan)
{
  struct proc *p = myproc();
  acquire(&p->lock);
  if (chan == 0)
    panic("sleep_prepare: zero chan");
  p->chan = chan;
  release(&p->lock);
}
void
sleep(void)
```

```
{
  struct proc *p = myproc();
  acquire(&p->lock);
  if (p->chan != 0) {
    p->state = SLEEPING;
    sched();
  }
  release(&p->lock);
}
void
wakeup(void *chan)
{
  struct proc *p;
  for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->chan == chan) {
      p->chan = 0;
      if (p->state == SLEEPING) {
        p->state = RUNNABLE;
      }
    }
    release(&p->lock);
  }
}
```

이 저장소는 교과서의 sleep(chan, lock) API가 아니다. 조건 lock을 잡은 상태에서 sleep_prepare(chan)으로 채널을 먼저 등록하고 lock을 푼 다음 sleep()한다. 그 사이 wakeup이 오면 chan=0으로 바뀌므로 sleep()은 실제로 잠들지 않는다. 이것이 lost wakeup을 막는 핵심 순서다.

04 · Sv39와 메모리 코드

가상주소 손계산과 실제 xv6 walk/page fault 연결

VA를 4개 조각으로 나눈다

Sv39, PGSIZE=4096(2^12). VA의 bits 38:30=VPN[2], 29:21=VPN[1], 20:12=VPN[0], 11:0=offset. 각 VPN은 9비트라 페이지 테이블 한 장의 PTE는 512개다. satp는 최상위 페이지 테이블을 가리킨다.

페이지 크기·PTE 비트·인덱스 매크로 | pa3:kernel/riscv.h L389-417

```
#define PGSIZE 4096
#define PGSHIFT 12
#define PGROUNDUP(sz) (((sz) + PGSIZE - 1) & ~(PGSIZE - 1))
#define PGROUNDDOWN(a) (((a)) & ~(PGSIZE - 1))
#define PTE_V (1L << 0)
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4)
#define PA2PTE(pa) ((((uint64)pa) >> 12) << 10)
#define PTE2PA(pte) (((pte) >> 10) << 12)
#define PTE_FLAGS(pte) ((pte) & 0x3FF)
#define PXMASK 0x1FF
#define PXSHIFT(level) (PGSHIFT + (9 * (level)))
#define PX(level, va) ((((uint64)(va)) >> PXSHIFT(level)) &
    PXMASK)
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
```

walk: level 2 → 1 → 0 | pa3:kernel/vm.c L98-116

```
pte_t *
walk(pagetable_t pagetable, uint64 va, int alloc)
{
  if (va >= MAXVA)
    panic("walk");
  for (int level = 2; level > 0; level--) {
    pte_t *pte = &pagetable[PX(level, va)];
    if (*pte & PTE_V) {
      pagetable = (pagetable_t)PTE2PA(*pte);
```

```
    } else {
      if (!alloc || (pagetable = (pde_t *)kalloc()) == 0)
        return 0;
      memset(pagetable, 0, PGSIZE);
      *pte = PA2PTE(pagetable) | PTE_V;
    }
  }
  return &pagetable[PX(0, va)];
}
```

walk는 중간 PTE가 V=1이면 다음 페이지 테이블의 물리주소로 내려간다. 없고 alloc=1이면 새 페이지 테이블 페이지를 만들고 PTE_V를 세운다. 마지막에는 level 0 PTE 주소를 돌려준다. 리프가 실제 물리 페이지를 가리키는지와 권한은 이후 검사다.

mappages: leaf PTE 생성 | pa3:kernel/vm.c L147–175

```
mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64
    pa, int perm)
{
  uint64 a, last;
  pte_t *pte;
  if ((va % PGSIZE) != 0)
    panic("mappages: va not aligned");
  if ((size % PGSIZE) != 0)
    panic("mappages: size not aligned");
  if (size == 0)
    panic("mappages: size");
  a = va;
  last = va + size - PGSIZE;
  for (;;) {
    if ((pte = walk(pagetable, a, 1)) == 0)
      return -1;
    if (*pte & PTE_V)
      panic("mappages: remap");
    *pte = PA2PTE(pa) | perm | PTE_V;
    if (a == last)
      break;
    a += PGSIZE;
    pa += PGSIZE;
  }
  return 0;
```

```
}
```

mappages는 시작 VA·크기가 페이지 정렬인지 검사하고, 각 페이지마다 PA2PTE(pa)|perm|PTE_V로 매핑한다. 이미 V인 PTE라면 remap panic이다. 일반 user 페이지에는 PTE_U가 필요하지만 TRAPFRAME/TRAMPOLINE 매핑에는 PTE_U가 없다.

기출형 계산 예

VA 0x0000000040001234 → offset=0x234, VPN[0]=1, VPN[1]=0, VPN[2]=1. leaf PTE가 PPN=0x80023을 주면 PA=(0x80023 ⟨⟨ 12)|0x234 = 0x80023234. 실제 하드웨어 접근은 V/U/R/W/X 권한 검사를 통과해야 한다.

copyin/out은 페이지 경계를 쪼갠다

copyout/copyin의 walkaddr와 페이지별 복사 | pa3:kernel/vm.c L345-376, 383-405

```
copyout(pagetable_t pagetable, uint64 psz, uint64 dstva, char
    *src, uint64 len)
{
  uint64 n, va0, pa0;
  pte_t *pte;
  while (len > 0) {
    va0 = PGROUNDDOWN(dstva);
    if (va0 >= MAXVA)
      return -1;
    pa0 = walkaddr(pagetable, va0);
    if (pa0 == 0) {
      if ((pa0 = vmfault(pagetable, psz, va0, 0)) == 0) {
        return -1;
      }
    }
    pte = walk(pagetable, va0, 0);
    if ((*pte & PTE_W) == 0)
      return -1;
    n = PGSIZE - (dstva - va0);
    if (n > len)
      n = len;
    memmove((void *)(pa0 + (dstva - va0)), src, n);
```

```
    len -= n;
    src += n;
    dstva = va0 + PGSIZE;
  }
  return 0;
    ...
copyin(pagetable_t pagetable, uint64 psz, char *dst, uint64
    srcva, uint64 len)
{
  uint64 n, va0, pa0;
  while (len > 0) {
    va0 = PGROUNDDOWN(srcva);
    pa0 = walkaddr(pagetable, va0);
    if (pa0 == 0) {
      if ((pa0 = vmfault(pagetable, psz, va0, 1)) == 0) {
        return -1;
      }
    }
    n = PGSIZE - (srcva - va0);
    if (n > len)
      n = len;
    memmove(dst, (void *)(pa0 + (srcva - va0)), n);
    len -= n;
    dst += n;
    srcva = va0 + PGSIZE;
  }
  return 0;
}
```

커널이 user pointer를 임의로 직접 역참조하지 않고, user pagetable에서 각 VA 페이지를 물리주소로 번역해 복사한다. copyout은 PTE_W도 검사한다. src/dst가 페이지 경계를 넘으면 다음 페이지에서 다시 walkaddr를 한다.

기본 sbrk는 eager, sbrklazy만 lazy

두 사용자 API가 다른 모드를 전달 | pa3:user/ulib.c L167–176

```
char *
sbrk(int n)
{
  return sys_sbrk(n, SBRK_EAGER);
```

```
}
char *
sbrklazy(int n)
{
  return sys_sbrk(n, SBRK_LAZY);
```

sys_sbrk: LAZY에서는 sz만 증가 | pa3:kernel/sysproc.c L39–64

```
uint64
sys_sbrk(void)
{
  uint64 addr;
  int t;
  int n;
  argint(0, &n);
  argint(1, &t);
  addr = myproc()->sz;
  if (t == SBRK_EAGER || n < 0) {
    if (growproc(n) < 0) {
      return -1;
    }
  } else {
    if (addr + n < addr)
      return -1;
    if (addr + n > TRAPFRAME)
      return -1;
    myproc()->sz += n;
  }
  return addr;
```

vmfault: 유효한 lazy 범위 첫 접근에 할당 | pa3:kernel/vm.c L454–478

```
uint64
vmfault(pagetable_t pagetable, uint64 psz, uint64 va, int read)
{
  uint64 mem;
  if (va >= psz)
    return 0;
  va = PGROUNDDOWN(va);
  if (ismapped(pagetable, va)) {
    return 0;
  }
  mem = (uint64)kalloc();
  if (mem == 0)
```

```
    return 0;
  memset((void *)mem, 0, PGSIZE);
  if (mappages(pagetable, va, PGSIZE, mem, PTE_W | PTE_U |
    PTE_R) != 0) {
    kfree((void *)mem);
    return 0;
  }
  return mem;
}
```

lazy 요청은 주소 공간 크기 sz만 먼저 키운다. 첫 load/store가 미매핑 페이지에 닿으면 usertrap의 scause 13/15 경로가 vmfault를 호출해 0으로 채운 페이지를 매핑한다. 이 구현의 vmfault(read 인자)는 본문에서 권한을 나누는 데 쓰이지 않는다.

fork의 uvmcopy는 이미 존재하는 페이지만 복사하고 lazy hole은 건너뛴다.

05 · PA1: 부팅 출력

이번 학기 제출 브랜치 pa1

요구: 부팅 화면에 SNUOS2026과 학번·이름을 표시한다. 코드의 핵심은 위치다. 아직 user process가 없는 hart 0의 kernel/main.c에서 consoleinit/printkinit 뒤, kinit·userinit 앞에 printk를 둔다.

hart 0의 출력 시점과 다음 초기화 | pa1:kernel/main.c L13-21, 36-43

```
  if (cpuid() == 0) {
    consoleinit();
    printkinit();
    printk("\n");
    printk("xv6 kernel is booting\n");
    printk("\n");
    printk("SNUOS2026\n");
    printk("\n");
    ...
    kinit();
    kvminit();
    kvminithart();
    procinit();
    trapinit();
    trapinithart();
    plicinit();
```

- 왜 hart 0? 전역 초기화와 부팅 메시지를 한 번만 실행하기 위해 cpuid()==0 분기를 쓴다. 다른 hart는 started를 기다린다.
- 왜 user printf가 아닌가? user address space나 sys_write 경로가 준비되기 전이다.
- Makefile의 STUDENTID는 제출물 식별값이며 출력 로직과 역할이 다르다.

06 · PA2: U→S→M의 두 번의 ecall

이번 학기 제출 브랜치 pa2 · nenter 23 / getpmpaddr 24 / getpmpcfg 25

nenter: 무엇을 세는가

수용된 개행만 누적 | pa2:kernel/console.c L147-151, 170-185, 188-194

```
uint nenter_count;
void
consoleintr(int c)
{
    ...
    }
    break;
  default:
    if (c != 0 && cons.e - cons.r < INPUT_BUF_SIZE) {
      c = (c == '\r') ? '\n' : c;
      if (c == '\n') nenter_count++;
      consputc(c);
      cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
      if (c == '\n' || c == C('D') || cons.e - cons.r ==
    INPUT_BUF_SIZE) {
    ...
        wakeup(&cons.r);
      }
    }
    break;
  }
  release(&cons.lock);
```

누적값 반환 | pa2:kernel/sysproc.c L115-120

```
extern uint nenter_count;
uint64 sys_nenter(void) {
  return nenter_count;
}
```

consoleintr의 default 분기에서 입력 버퍼에 공간이 있고 c가 0이 아닐 때, CR을 LF로 바꾼 뒤 LF면 카운트한다. Ctrl-U/Backspace로 지운 문자나 버퍼가 가득 차 수용되지 않은 Enter는 포함되지 않는다. 증가 쪽은 cons.lock 내부, 읽는 sys_nenter에는 별도 lock이 없다.

PMP 조회의 두 번째 trap

mtvec 등록과 medeleg bit 9 해제 | pa2:kernel/start.c L29-37

```
  w_satp(0);
  w_mtvec((uint64) machinevec);
  w_medeleg(0xfdff);
  w_mideleg(0xffff);
```

인덱스 0..3 검사 후 M-mode 래퍼 | pa2:kernel/sysproc.c L122-144

```
extern uint64 mcall_getpmpaddr(int n);
extern uint64 mcall_getpmpcfg(int n);
uint64 sys_getpmpaddr(void) {
  int n;
  argint(0, &n);
  if (n < 0 || n > 3) return -1;
  return mcall_getpmpaddr(n);
}
uint64 sys_getpmpcfg(void) {
  int n;
  argint(0, &n);
  if (n < 0 || n > 3) return -1;
  return (int) mcall_getpmpcfg(n);
}
```

0xfdff = 0xffff & ~0x200. medeleg bit 9(S-mode ecall)를 끄므로 S-mode의 ecall이 machinevec로 간다. 사용자 ecall(cause 8)은 기존 S-mode syscall로 간다. n〈0 또는 n〉3이면 sysproc이 -1을 반환하므로 M-mode 호출까지 가지 않는다.

mepc+4, a7 분기, 주소 변환, 래퍼 | pa2:kernel/kernelvec.S L66-80, 110-114, 157-168

```
.globl machinevec
machinevec:
```

```
        csrr t0, mepc
        addi t0, t0, 4
        csrw mepc, t0
        li t0, 24
        beq a7, t0, handle_getpmpaddr
        li t0, 25
        beq a7, t0, handle_getpmpcfg
        li a0, -1
        mret
    ...
process_pmpaddr:
        slli a0, a0, 2
        li t0, 0x00FFFFFFFFFFFFFF
        and a0, a0, t0
        mret
    ...
mcall_getpmpaddr:
        li a7, 24
        ecall
        ret
.globl mcall_getpmpcfg
mcall_getpmpcfg:
        li a7, 25
        ecall
        ret
```

래퍼가 a7=24/25, a0=n을 놓고 ecall한다. machinevec는 mepc를 4 올려 같은 ecall 반복을 막는다. getpmpaddr는 pmpaddr[n] CSR 원값을 왼쪽 2비트 이동하고 0x00FFFFFFFFFFFFFF로 마스크한다.

pmpcfg0의 n번째 바이트 추출 | pa2:kernel/kernelvec.S L116-127, 128-152

```
handle_getpmpcfg:
        li t0, 0
        beq a0, t0, read_pmpcfg0
        li t0, 1
        beq a0, t0, read_pmpcfg1
        li t0, 2
        beq a0, t0, read_pmpcfg2
        li t0, 3
        beq a0, t0, read_pmpcfg3
        li a0, -1
```

```
        mret
    ...
read_pmpcfg0:
        csrr a0, pmpcfg0
        li t0, 0x00000000000000FF
        and a0, a0, t0
        mret
read_pmpcfg1:
        csrr a0, pmpcfg0
        li t0, 0x000000000000FF00
        and a0, a0, t0
        srli a0, a0, 8
        mret
read_pmpcfg2:
        csrr a0, pmpcfg0
        li t0, 0x0000000000FF0000
        and a0, a0, t0
        srli a0, a0, 16
        mret
read_pmpcfg3:
        csrr a0, pmpcfg0
        li t0, 0x00000000FF000000
        and a0, a0, t0
        srli a0, a0, 24
```

pmpcfg0는 8비트 설정 8개를 담는다. n=0..3에는 각각 bits 7:0, 15:8, 23:16, 31:24를 마스크·우측 이동한다. 예: 0xA1B2C3D4에서 n=2면 0xB2.

실제 초기값을 손으로 계산

PA2의 pmp.c는 pmpaddr0=0x3FFFFFFFFFFFFF, pmpcfg0=0xF로 설정한다. 주소 반환은 (원값〈〈2) & 0x00FFFFFFFFFFFFFF = 0x00FFFFFFFFFFFFFC. 설정 n=0은 0x0F로 R/W/X와 TOR 비트가 켜져 있다. n=1..3의 기본 예시는 주소·설정 0이다.

M-mode machinevec는 이 제출의 짧은 서비스 경로이며, 모든 M-mode 예외·인터럽트를 일반적으로 처리하는 저장/복원 루틴이 아니다.

07 · PA3: EDF를 코드로 따라가기

이번 학기 제출 브랜치 pa3 · sched_setattr 30 / sched_yield 31

어떤 프로세스가 RT인가

runtime, period, deadline | pa3:kernel/proc.h L92–94

```
  int runtime;
  int period;
  uint deadline;
```

allocproc에서 0 초기화 | pa3:kernel/proc.c L125–131

```
  p->pid = allocpid();
  p->state = USED;
  p->runtime = 0;
  p->period = 0;
  p->deadline = 0;
```

syscall 검증과 양보 | pa3:kernel/sysproc.c L116–140

```
sys_sched_setattr(void)
{
  int pid;
  int runtime;
  int period;
  argint(0, &pid);
  argint(1, &runtime);
  argint(2, &period);
  if (runtime >= period) return -1;
  if (runtime <= 0) return -1;
  if (period <= 0) return -1;
  return ksched_setattr(pid, runtime, period);
}
uint64
sys_sched_yield(void)
{
  ksched_yield();
  return 0;
}
```

runtime>0, period>0, runtime<period이어야 한다. pid=0은 자기 자신을 가리킨다. fork 자식은 allocproc에서 RT 필드가 0이므로 부모의 RT 속성을 자동 상속하지 않는다.

deadline = 현재 ticks + period | pa3:kernel/proc.c L767-793

```
ksched_setattr(int pid, int runtime, int period) {
  struct proc *p;
  uint now;
  if (pid == 0) pid = myproc()->pid;
  acquire(&tickslock);
  now = ticks;
  release(&tickslock);
  for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->pid == pid && p->state != UNUSED) {
      p->runtime = runtime;
      p->period = period;
      p->deadline = now + period;
      release(&p->lock);
      return 0;
    }
    release(&p->lock);
  }
  return -1;
```

job 완료가 일반 yield와 다른 이유

RT면 SLEEPING, 일반이면 RUNNABLE | pa3:kernel/proc.c L796-811

```
void ksched_yield(void) {
  struct proc *p = myproc();
  acquire(&p->lock);
  if (p->runtime > 0 && p->period > 0) {
    p->state = SLEEPING;
  } else {
    p->state = RUNNABLE;
  }
  sched();
  release(&p->lock);
}
```

일반 sched_yield는 CPU만 양보한다. RT의 sched_yield는 이번 job의 완료 선언이라 SLEEPING이 된다. sleep(chan)과 달리 scheduler가 now〉=deadline이면 직접 RUNNABLE로 만든다. runtime을 실제로 차감하거나 실행을 강제로 막는 코드는 없다.

매 순회 EDF 후보를 고른다

ticks 읽기, 주기 재활성화, 최소 deadline | pa3:kernel/proc.c L449-483

```
    intr_on();
    intr_off();
    uint now;
    uint best_deadline = 0;
    struct proc *best = 0;
    int best_pid = 0;
    acquire(&tickslock);
    now = ticks;
    release(&tickslock);
    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if (p->period > 0 && p->runtime > 0) {
        if (p->state == SLEEPING && now >= p->deadline) {
          p->deadline += p->period;
          p->state = RUNNABLE;
        }
        if (p->state == RUNNABLE) {
          if (best== 0 || p->deadline < best_deadline ||
(p->deadline == best_deadline && best != previous && (p ==
previous || p->pid < best_pid))) {
            best_deadline = p->deadline;
            best = p;
            best_pid = p->pid;
          }
        }
      }
      release(&p->lock);
    }
```

잠든 RT가 now⟩=deadline이면 deadline += period를 한 번 실행하고 깨운다. RUNNABLE RT 가운데 가장 이른 절대 deadline을 고른다. 동률이면 명세의 '현재 프로세스 우선'을 구현의 previous 포인터로 판단하고, 그 외에는 PID가 작은 쪽을 고른다.

RT 우선, 없으면 원형 RR, 없으면 wfi | pa3:kernel/proc.c L486–531

```
    if (best != 0) {
      acquire(&best->lock);
      best->state = RUNNING;
      c->proc = best;
      swtch(&c->context, &best->context);
      mycpu()->intena = 0;
      previous = best;
      c->proc = 0;
      release(&best->lock);
      continue;
    }
    int found = 0;
    for (int i = 0; i < NPROC; i++) {
      int index = (next + i) % NPROC;
      p = &proc[index];
      acquire(&p->lock);
      if (p->state == RUNNABLE) {
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        mycpu()->intena = 0;
        next = (index + 1) % NPROC;
        previous= p;
        c->proc = 0;
        found = 1;
        release(&p->lock);
        break;
      }
      release(&p->lock);
    }
    if (found == 0) {
      asm volatile("wfi");
    }
```

RT 후보가 있으면 일반 프로세스는 선택되지 않는다. 없으면 next 위치부터 원형으로 RUNNABLE을 찾아 실행한다. 타이머 인터럽트는 yield()를 거쳐 재선택을 일으키므로 새 RT 작업이 생겨도 즉시 선점하는 것은 아니며 다음 재스케줄링 지점에서 반영된다.

기출형 시간표: task1

tick 0에 P1=(1,3), P2=(1,4) 등록. 초기 deadline은 3·4. 둘 다 매 job 끝에 sched_yield한다고 가정하면 [0,1) P1, [1,2) P2, [2,3) idle/일반, [3,4) P1, [4,5) P2, [5,6) idle/일반, [6,7) P1. tick 3에 P1을 깨울 때 deadline은 6, tick 4의 P2는 8이 된다.

task1 사용률 1/3+1/4=7/12 = 약 0.583. task2는 1/8+2/5+4/10=0.925. U≤1은 이상적인 단일 CPU EDF의 실행 가능성 기준이다. 제출 코드는 admission test나 runtime 초과 차단을 구현하지 않으며, 과제는 실제 실행시간〈runtime·RT I/O 없음·CPUS=1을 가정한다.

코드 경계와 함정

- scheduler는 RT 후보를 조사한 뒤 lock을 풀고 다시 best->lock을 잡지만 상태를 재확인하지 않는다. 다중 CPU로 일반화할 때 경합 위험이 있으나 과제는 단일 CPU를 가정한다.
- 여러 period를 놓친 경우 deadline에 period를 한 번만 더한다. 모든 과거 deadline을 while로 건너뛰는 구현이 아니다.
- PA3는 PA2를 합친 브랜치가 아니다. PA3의 start.c에는 PA2의 machinevec/medeleg 수정이 없다.

08 · 예상 문제와 해설

기출의 질문 방식을 참고해 이번 xv6/PA 코드 기준으로 새로 만든 문제

01. U-mode ecall 직후 uservec 첫 csrw 이전의 a0, satp, sp, sepc는 무엇을 뜻하는가?

해설 | a0는 사용자 값, satp는 사용자 페이지 테이블, sp는 사용자 스택 값, sepc는 ecall 명령 주소다. 하드웨어가 커널 스택·커널 satp로 바꾸지 않는다.

02. csrw sscratch,a0 다음에 li a0,TRAPFRAME을 하는 이유는?

해설 | 사용자 a0를 sscratch에 임시 보관해 a0를 trapframe 포인터로 재사용하기 위해서다. 뒤에 csrr t0,sscratch; sd t0,112(a0)로 원래 값을 저장한다.

03. U-mode ecall과 timer interrupt의 sepc 처리 차이는?

해설 | ecall은 같은 명령 반복을 피하려고 epc+=4 한다. 타이머는 중단 명령을 재개해야 하므로 이 경로에서 +4 하지 않는다.

04. 사용자 시스템 콜의 번호가 a0라는 주장과, 결과가 a7이라는 주장을 각각 판별하라.

해설 | 둘 다 거짓. 번호는 a7, 첫 인자는 a0, 결과는 trapframe->a0에 써서 userret이 복원한다.

05. TRAMPOLINE을 사용자·커널 페이지 테이블의 같은 VA에 두는 이유는?

해설 | uservec/userret이 satp를 바꾸는 바로 그 순간에도 연속해서 코드를 실행해야 하기 때문이다. 사용자 모드에서 자유롭게 실행하도록 PTE_U를 주는 것은 아니다.

06. per-process, per-core, per-system을 분류하라: trapframe, cpu.context, ticks, syscalls[], pagetable.

해설 | trapframe·pagetable은 프로세스별, cpu.context는 CPU별, ticks·syscalls[]는 시스템 전체다.

07. fork 직후 같은 소스 줄에서 부모와 자식의 반환값이 다른 이유는?

해설 | 부모는 kfork가 PID를 반환하고 syscall이 부모 a0에 기록한다. 자식은 복제된 trapframe의 a0를 0으로 고친다.

08. swtch가 trapframe 대신 저장하는 것은 무엇인가?

해설 | 커널 문맥의 ra/sp/s0–s11 같은 callee-saved 레지스터다. U→S 시 사용자 레지스터 저장은 trampoline의 trapframe 경로다.

09. wakeup이 sleep_prepare와 sleep 사이에 오면 왜 영원히 잠들지 않는가?

해설 | sleep_prepare가 채널을 먼저 등록했고 wakeup이 chan을 0으로 지운다. 이어 sleep()은 채널이 없음을 보고 SLEEPING으로 바꾸지 않는다.

10. Sv39 VA 0x40001234의 VPN[2:0]과 offset은? leaf PPN=0x80023이면 PA는?

해설 | VPN[2]=1, VPN[1]=0, VPN[0]=1, offset=0x234. PA=(0x80023〈〈12)|0x234=0x80023234.

11. sbrk와 sbrklazy 중 실제 페이지를 즉시 늘리는 것은?

해설 | sbrk가 SBRK_EAGER를 넘긴다. sbrklazy는 sz만 늘리고 첫 접근의 유효한 fault에서 vmfault가 페이지를 할당한다.

12. PA1 부팅 메시지가 hart마다 중복되지 않는 이유는?

해설 | main의 cpuid()==0 분기 안에 printk가 있다. 다른 hart는 전역 초기화 완료를 기다리고 자기 설정만 한다.

13. PA2에서 Enter를 쳤지만 nenter가 증가하지 않는 경우 하나와 이유는?

해설 | 예를 들어 콘솔 입력 버퍼에 공간이 없으면 default 분기의 수용 조건을 통과하지 못해 LF 검사·증가가 실행되지 않는다.

14. PA2 getpmpaddr(0) 기본값과 근거는?

해설 | pmpaddr0 원값 0x3FFFFFFFFFFFFF를 2비트 왼쪽 이동하고 56비트 마스크하면 0x00FFFFFFFFFFFFFC다.

15. PA2의 S-mode ecall에서 medeleg bit 9, a7, mepc는 각각 왜 중요한가?

해설 | bit 9를 0으로 두어 M-mode에 도착하게 한다. a7=24/25로 서비스를 구분한다. mepc+=4가 없으면 mret 뒤 ecall에 재진입한다.

16. PA3 sched_setattr(0,2,5)의 검증·대상·초기 deadline은?

해설 | 2〉0, 5〉0, 2〈5라 유효하다. pid=0은 자기 프로세스이며 deadline=등록 순간 ticks+5다.

17. PA3 RT와 일반 프로세스가 sched_yield 후 갖는 상태를 비교하라.

해설 | RT는 이번 job 완료를 뜻해 SLEEPING, 일반은 RUNNABLE이다. RT는 deadline 도달 시 scheduler가 깨운다.

18. deadline이 같은 RT 두 개에서 PID가 큰 쪽이 직전 실행 프로세스라면?

해설 | 구현의 previous가 그 후보라면 PID보다 우선한다. previous가 두 후보 중 없으면 작은 PID가 이긴다.

19. PA3에 runtime=1이면 timer 1회 후 자동으로 RT가 잠드는가?

해설 | 아니다. 제출 코드에는 runtime 잔량 계산이 없다. RT job의 완료는 그 프로세스의 sched_yield() 호출로 표현한다.

20. 기출의 'M software interrupt는 pending 비트를 지워야 한다'를 현재 PA2 machinevec에 그대로 적용할 수 있는가?

해설 | 아니다. 현재 machinevec의 원인은 동기 S-mode ecall이며 mepc+4가 핵심이다. 비동기 software interrupt의 pending 해제·재진입 문제와 분리해야 한다.

21. tick 0에 PA3의 P1=(1,3), P2=(1,4)를 등록했다. 각 job이 1 tick을 사용하고 완료를 알린다면 tick 0-10 실행 순서는?

해설 | [0,1) P1, [1,2) P2, [2,3) 일반/idle, [3,4) P1, [4,5) P2, [5,6) 일반/idle, [6,7) P1, [7,8) 일반/idle, [8,9) P2, [9,10) P1. 각 release 시 deadline에 period를 더하고 후보의 절대 deadline을 비교한다.

22. tick 5에 deadline 14인 P2가 실행 중인데 새 RT P3의 deadline이 10으로 설정됐다. 즉시 P3가 선점하는가?

해설 | 아니다. 이번 명세는 RT 생성 시 즉시 선점하지 않는다. 다음 timer interrupt 등 재스케줄링 지점에서 P3가 RUNNABLE이면 더 이른 deadline 10 때문에 선택된다. fork 직후 자식은 normal이므로 P3에 sched_setattr을 했다고 가정해야 한다.

09 · 마지막 3분 카드

한 줄씩 소리 내어 설명

- trap: 하드웨어는 모드/CSR/PC, trampoline은 레지스터·sp·satp, C는 원인 분기.
- syscall: a7 번호, a0–a5 인자, 반환 a0, U ecall의 epc+4.
- context: trapframe=사용자 레지스터; swtch context=커널 callee-saved 문맥.
- fork/wait: 부모 PID/자식 0; 자식 ZOMBIE를 부모 wait가 회수.
- Sv39: 9+9+9+12비트; 중간 PTE는 다음 표, leaf는 PA와 권한.
- PA2: console LF 카운트; medeleg bit9=0; M handler의 mepc+4; PMP 주소 〈〈2 / cfg 바이트.
- PA3: deadline=ticks+period; RT 최우선·최소 deadline; 완료 yield→SLEEPING; 일반 RR.

자료 출처와 적용 범위

코드: 로컬 Git pa1(c236869), pa2(9ccbcf4), pa3(f42c080). 명세: snu-csl/os-pa1, os-pa2, os-pa3(2026 가을). 출제 형식: 제공된 2023-2, 2024-1, 2024-2, 2025-2 중간고사. 예상 문제는 원문 복제가 아니라 새로 구성한 연습 문제다.

주의: 일반 xv6 코드는 pa3 브랜치에서 발췌했다. PA1·PA2·PA3는 서로 다른 과제 브랜치라 각 절의 기능이 한 커널에 모두 합쳐진 것으로 해석하지 않는다.